INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

Course: BVWS103-OWASP Top 10 Vulnerabilities And Exploitation Techniques

Student Name: Abubakari-Sadique Hamidu
Student ID: 2025/FWSD/11392

Instructor: Aminu Iddris (CCNA, CompTIA, CEH, OSCP, CISSP, CISM)

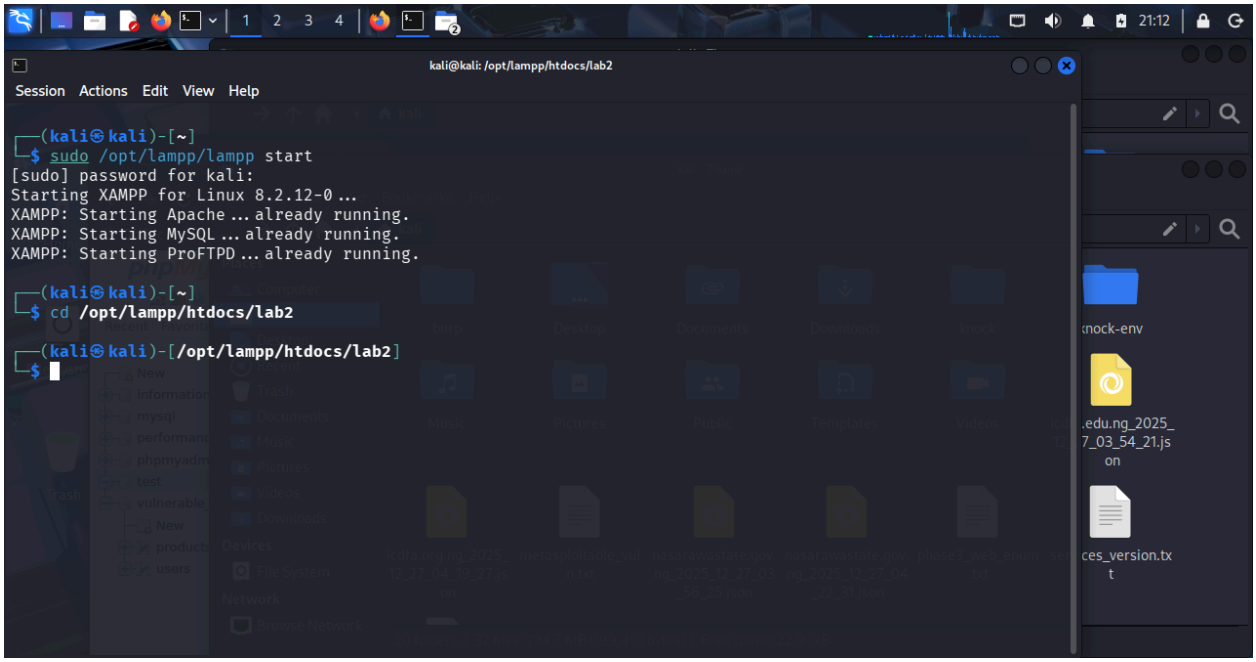
Title: HTML Injection

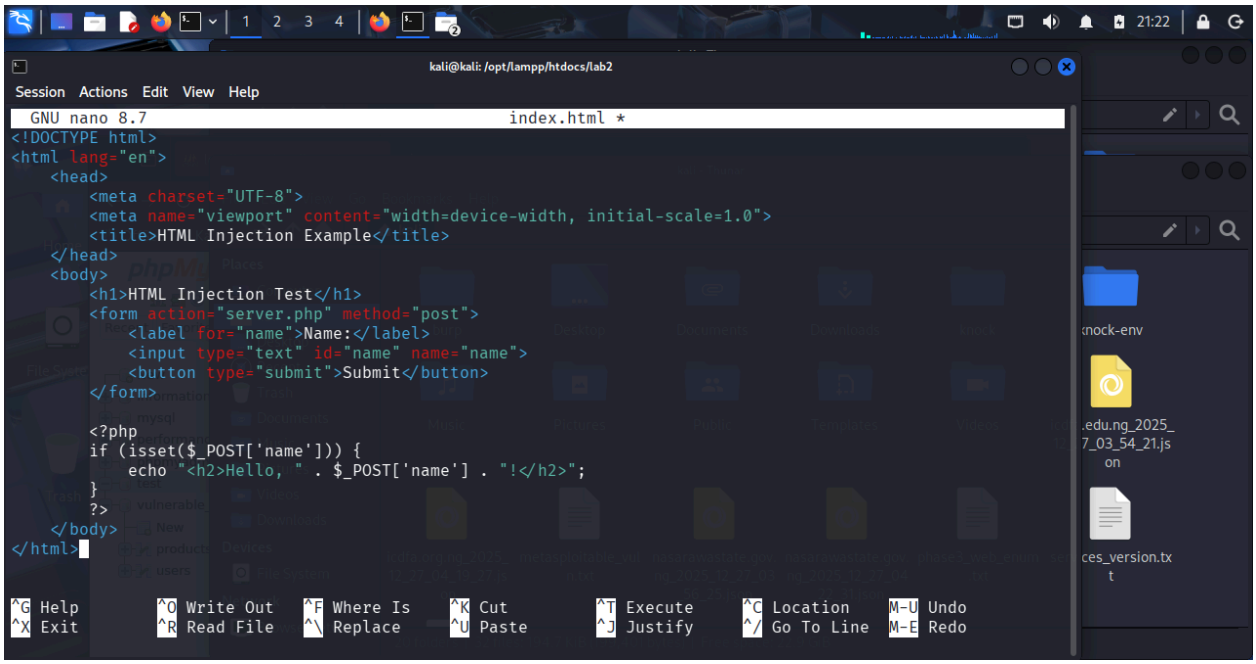
Objective

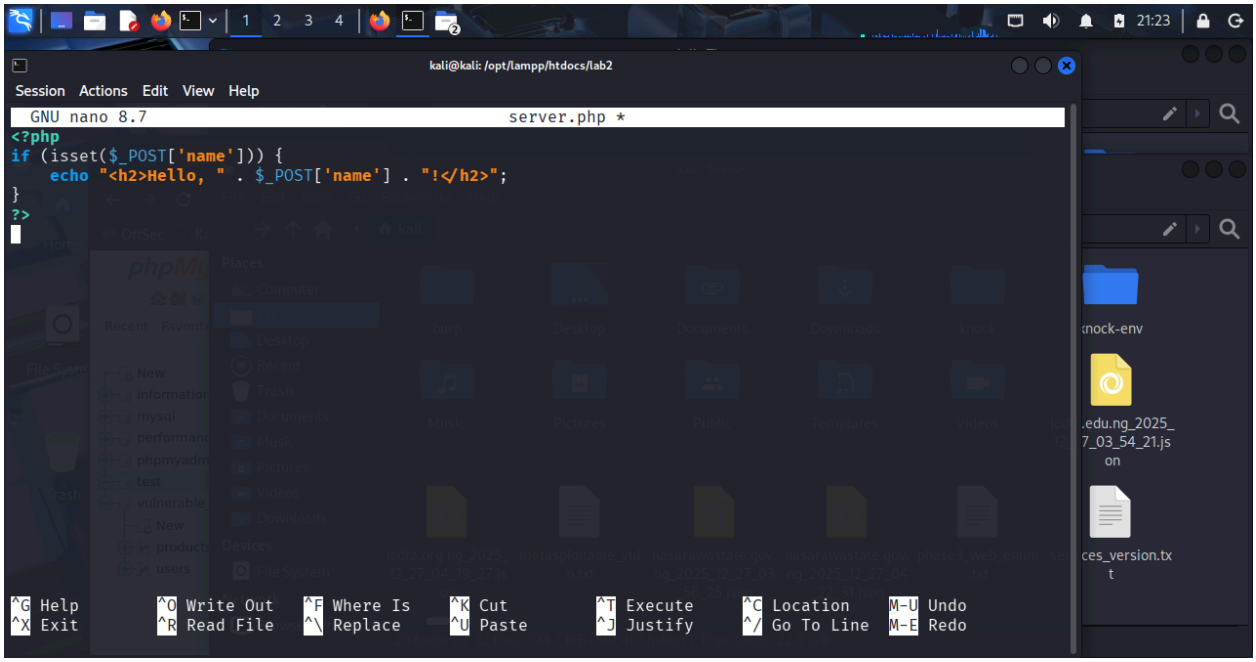
In this Lab, i simply learn how HTML injection works. I test an unsafe input flow. I apply input sanitization to stop script execution.

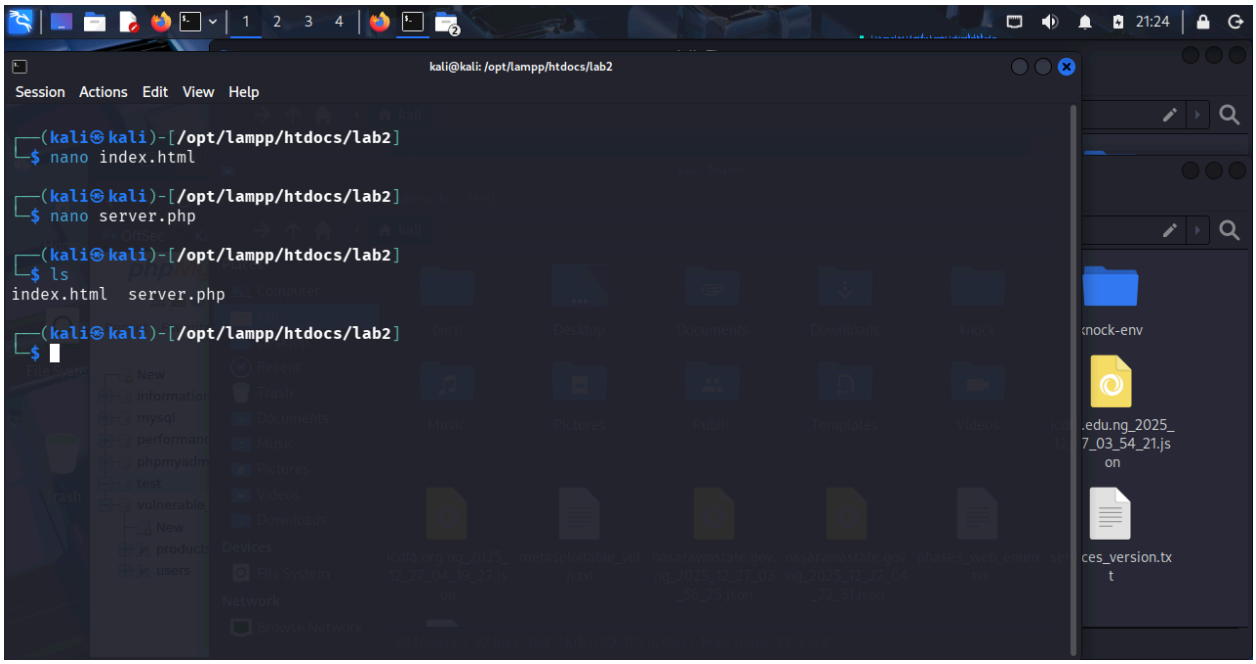
Setup

I create a folder named Lab2
I add two files. index.php and server.php.
I run the files through a local PHP server.


kali@kali: /opt/lampp/htdocs/lab2
Session Actions Edit View Help
┌──(kali㉿kali)-[~]
└─$ sudo /opt/lampp/lampp start
[sudo] password for kali:
Starting XAMPP for Linux 8.2.12-0 ...
XAMPP: Starting Apache ... already running.
XAMPP: Starting MySQL ... already running.
XAMPP: Starting ProFTPD ... already running.
┌──(kali㉿kali)-[~]
└─$ cd /opt/lampp/htdocs/lab2
┌──(kali㉿kali)-[/opt/lampp/htdocs/lab2]
└─$


kali@kali: /opt/lampp/htdocs/lab2
Session Actions Edit View Help
GNU nano 8.7 index.html *
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>HTML Injection Example</title>
</head>
<body>
<h1>HTML Injection Test</h1>
<form action="server.php" method="post">
<label for="name">Name:</label>
<input type="text" id="name" name="name">
<button type="submit">Submit</button>
</form>
<?php
if (isset($_POST['name'])) {
echo "<h2>Hello, " . $_POST['name'] . "!</h2>";
}
?>
</body>
</html>
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo





Vulnerable Implementation

The form in index.php sends user input to server.php.

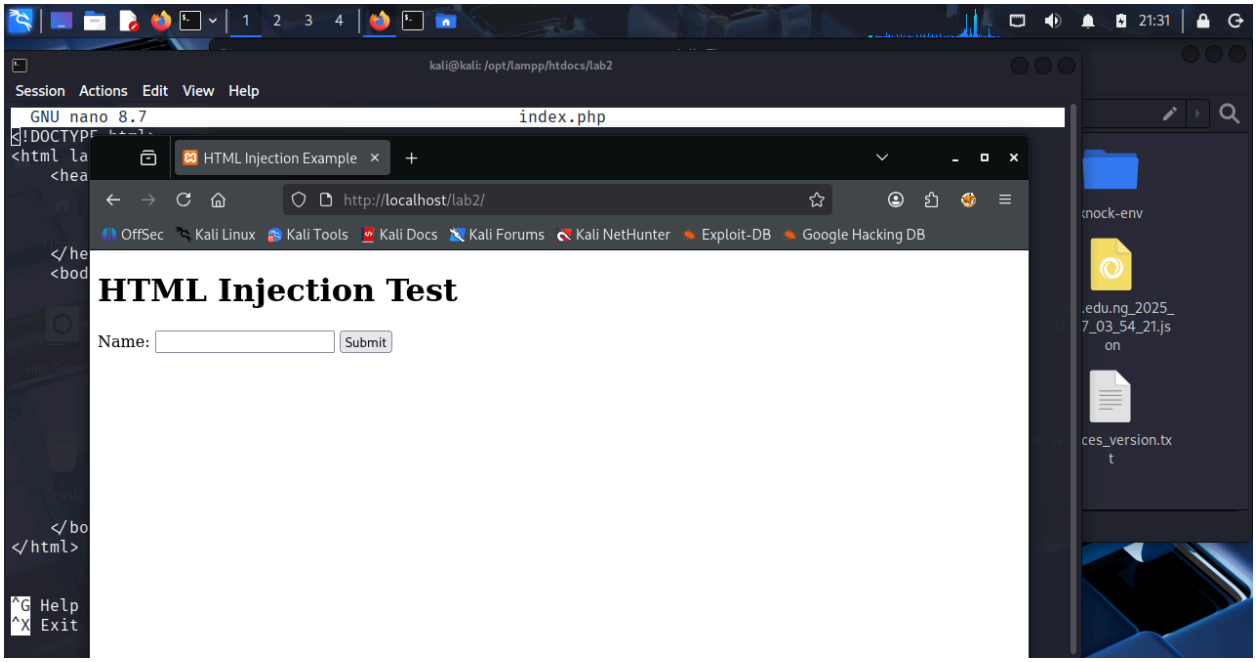
The server prints user input directly inside HTML.

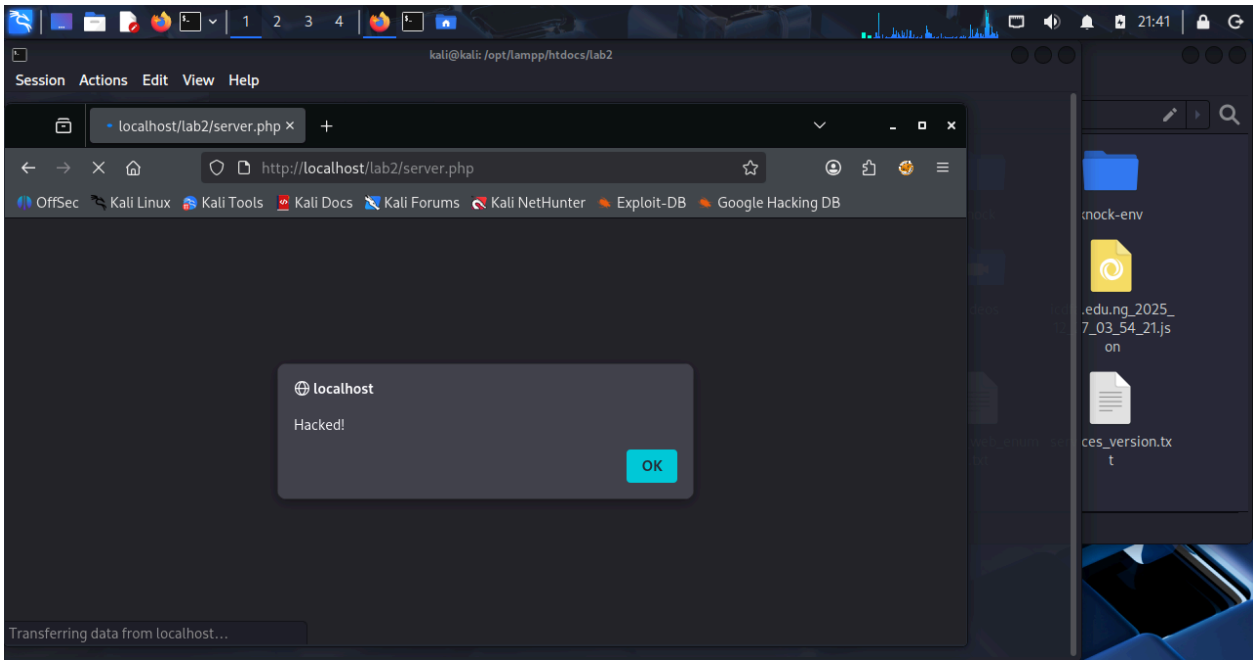
No filtering occurs.
The browser interprets input as code.

Exploitation

I enter this payload in the Name field.

I submit the form.
The browser executes the script.
An alert popup appears.



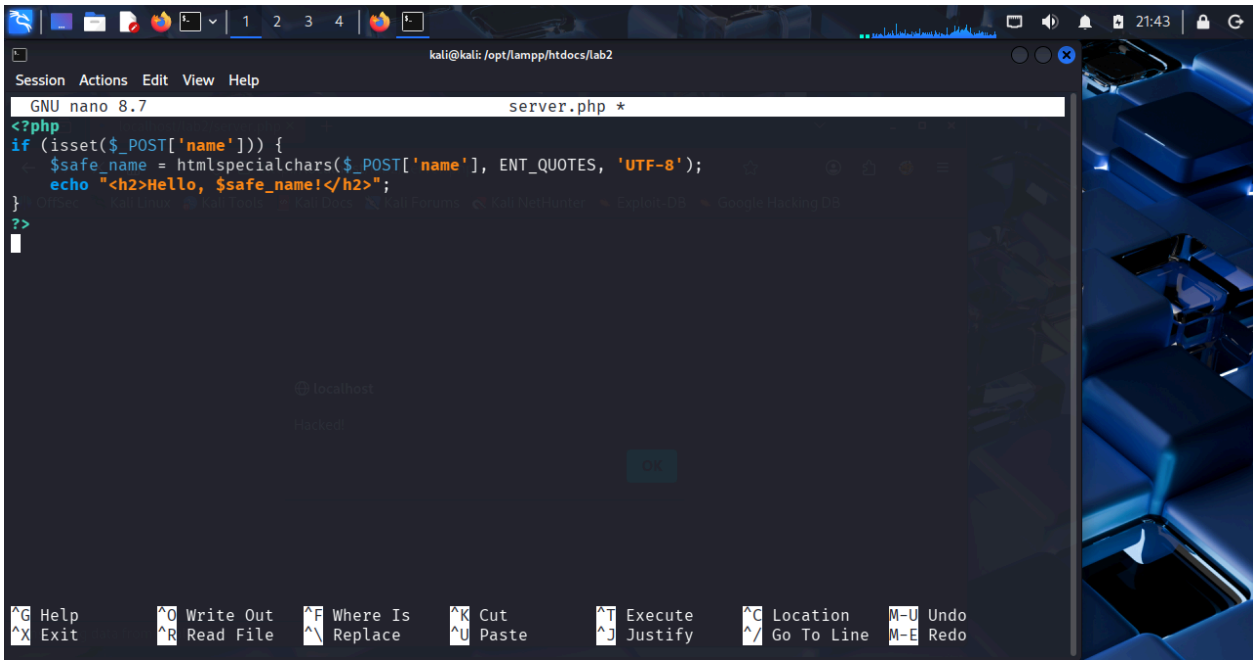


Impact

An attacker injects scripts.
The script runs in the user browser.
The attacker steals session data.
The attacker alters page content.
The attacker spreads malicious links.

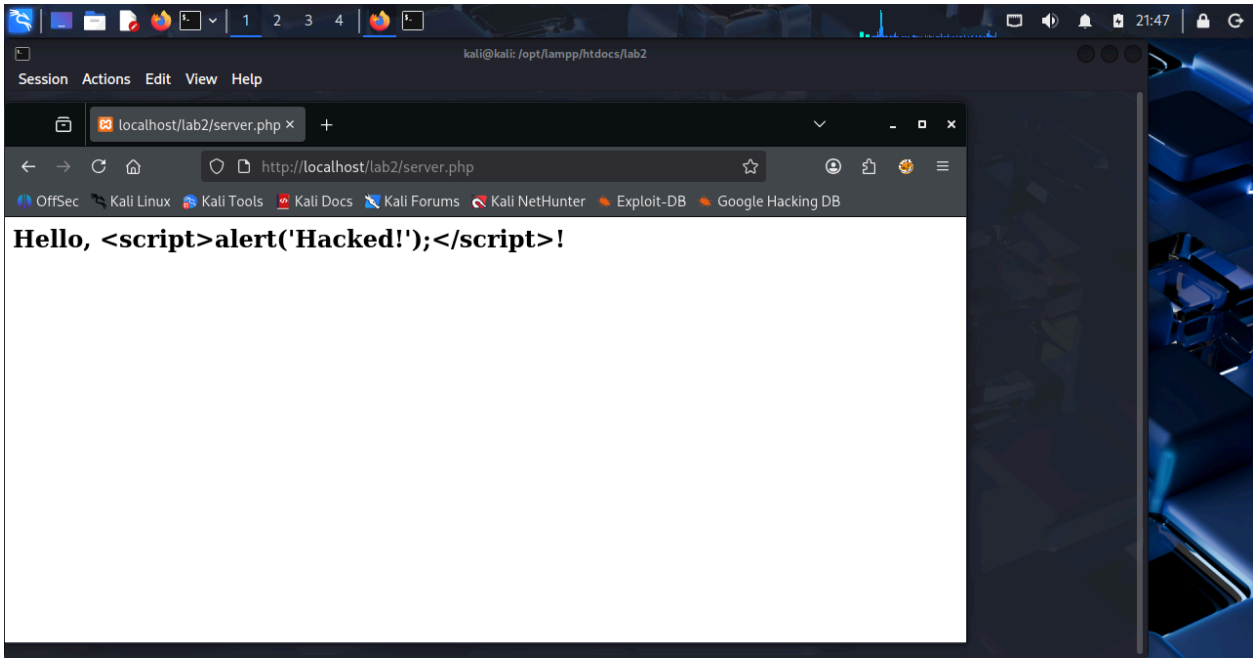
Mitigation

I sanitize input using htmlspecialchars.
Special characters convert to safe HTML entities.
The browser treats input as text.
Script execution stops.



Verification After Fix

I submit the same payload again.
No alert appears.
The script displays as plain text.



Lessons Learned

Treat all user input as unsafe.
Sanitized input before output.
I reduced HTML injection and XSS risk.
Protect users and application integrity.